MACHINE LEARNING COURSE CODE: CSE 4611

PRESENTED BY:

DR. AMRAN HOSSAIN

PROFESSOR

DEPARTMENT OF CSE, DUET

MACHINE LEARNING (ML)

- Machine Learning is a subfield of artificial intelligence (AI) that focuses on developing algorithms and models that enable computers to learn from data and make predictions or decisions without being explicitly programmed.
- It involves the study and construction of computer systems that can automatically improve and adapt their performance through experience.
- In Machine Learning, the learning process is driven by data rather than explicit instructions. The algorithms are designed to analyze and extract patterns, insights, and relationships from large datasets.
- By learning from these patterns, the machine learning models can generalize and make predictions or take actions on new, unseen data.
- The main goal of Machine Learning is to develop
  - ✓ computational models that can automatically learn from data and improve their performance over time without human intervention.

MACHINE LEARNING

- These models can be trained using various techniques such as supervised learning, unsupervised learning, semi-supervised learning, reinforcement learning, and deep learning.
- Machine Learning is a subfield of artificial intelligence (AI) that focuses on developing algorithms and models that enable computers to learn from data and make predictions or decisions without being explicitly programmed.
- It involves the study and construction of computer systems that can automatically improve and adapt their performance through experience.

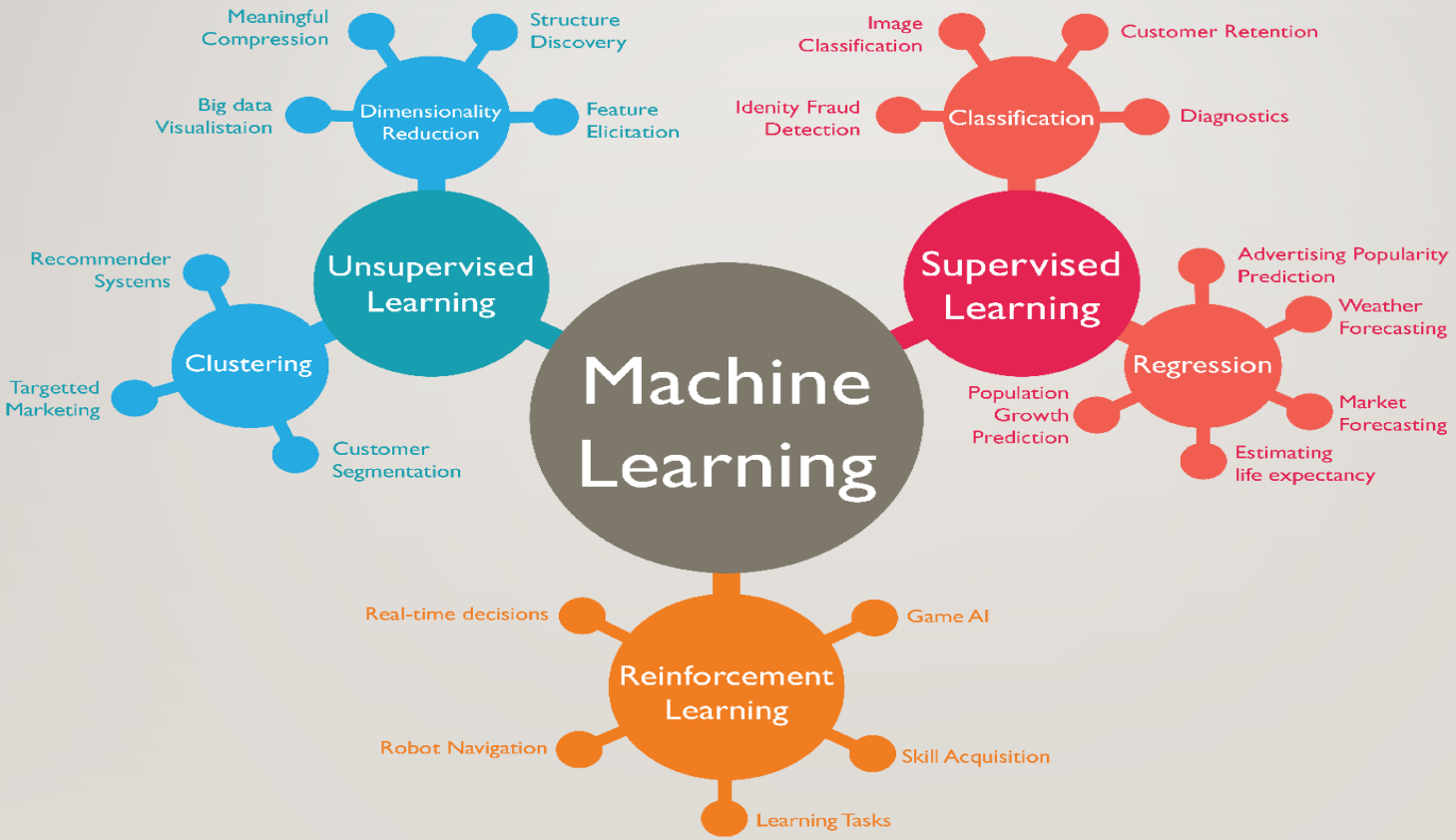
MACHINE LEARNING ALGORITHMS

- 1) Classical Learning
  - 1) Supervised Learning
    - Classification
    - Regression
  - 2)Unsupervised Learning
    - Clustering
    - Pattern Search
    - Dimension reduction
- 2)Reinforcement Learning
- 3) Ensemble Methods
- 4)Neural Nets and Deep Learning

MACHINE LEARNING ALGORITHMS

- 1) Classification
  - K-NN
  - Naïve Bayes
  - Support Vector Machine (SVM)
  - Decision Tree
  - Logistic Regression
- 2) Regression
  - Linear regression
  - Polynomial Regression
  - Ridge/Lasso Regression

APPLICATIONS OF ML



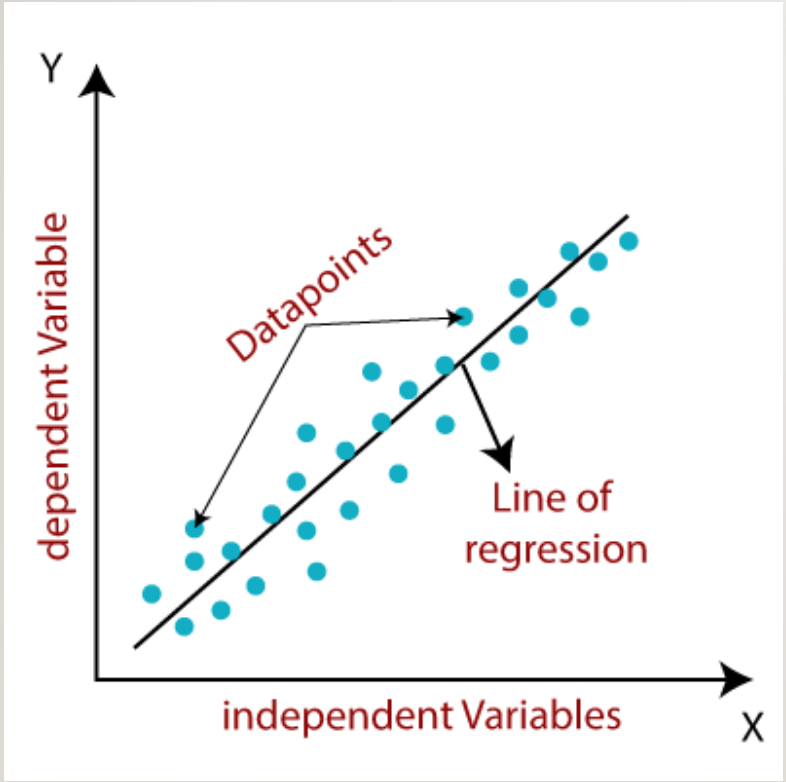
SUPERVISED LEARNING

- Supervised Learning: Supervised learning algorithms learn from labeled training data, where the desired output or target variable is known. The algorithm learns to map input features to the correct output based on the provided labels. The goal is to create a model that can accurately predict the output for new, unseen inputs. Supervised learning can be further divided into two sub-categories:
  - Classification: In classification tasks, the target variable is categorical or discrete. The algorithm learns to classify input data into predefined classes or categories. Examples include spam email detection, sentiment analysis, or image classification.
  - Regression: In regression tasks, the target variable is continuous, and the algorithm learns to predict a numeric value. The algorithm learns the relationship between input features and the corresponding numeric output. Examples include predicting house prices or estimating sales revenue.

LINEAR REGRESSION

- **Linear regression is an algorithm that provides a linear relationship between an independent variable and a dependent variable to predict the outcome of future events.**
- **It is a statistical method used in data science and machine learning for predictive analysis.**
- **Linear regression is one of the easiest and most popular Machine Learning algorithms. It is a statistical method that is used for predictive analysis. Linear regression makes predictions for continuous/real or numeric variables such as sales, salary, age, product price, etc.**
- **Linear regression algorithm shows a linear relationship between a dependent (y) and one or more independent (y) variables, hence called as linear regression. Since linear regression shows the linear relationship, which means it finds how the value of the dependent variable is changing according to the value of the independent variable.**

LINEAR REGRESSION



LINEAR REGRESSION

- Types of Linear Regression
  - Linear regression can be further divided into two types of the algorithm:
  - **Simple Linear Regression:**
    If a single independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Simple Linear Regression.
  - **Multiple Linear regression:**
    If more than one independent variable is used to predict the value of a numerical dependent variable, then such a Linear Regression algorithm is called Multiple Linear Regression.

LINEAR REGRESSION

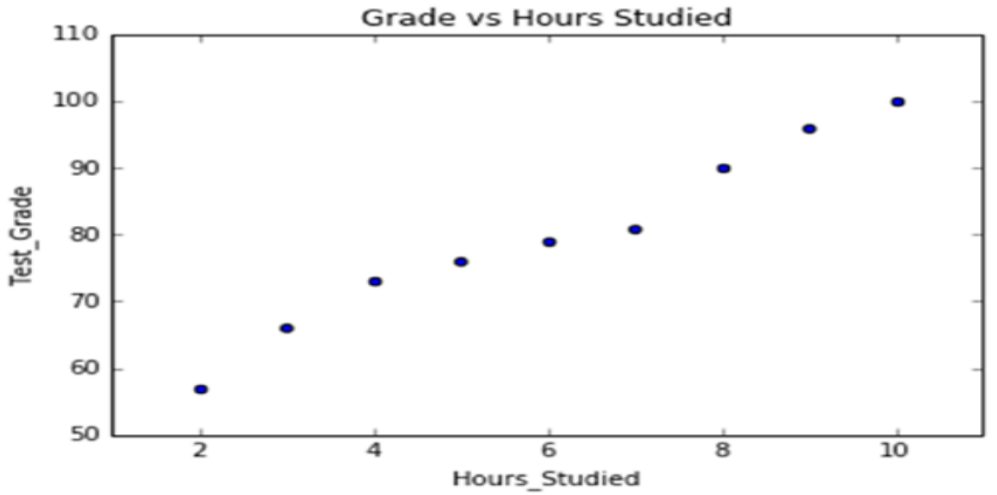
- Let's consider a use case where we have collected students' average test grade scores and their respective average number of study hours for the test for group of similar IQ students. See Listing 3-9.

```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

# Load data
df = pd.read_csv('Data/Grade_Set_1.csv')
print df

# Simple scatter plot
df.plot(kind='scatter', x='Hours_Studied', y='Test_Grade', title='Grade vs
Hours Studied')
# check the correlation between variables
Print df.corr()
# ---- output ----
   Hours_Studied  Test_Grade
0              2          57
1              3          66
2              4          73
3              5          76
```

LINEAR REGRESSION

- Let's consider a use case where we have collected students' average test grade scores and their respective average number of study hours for the test for group of similar IQ students. See Listing 3-9.

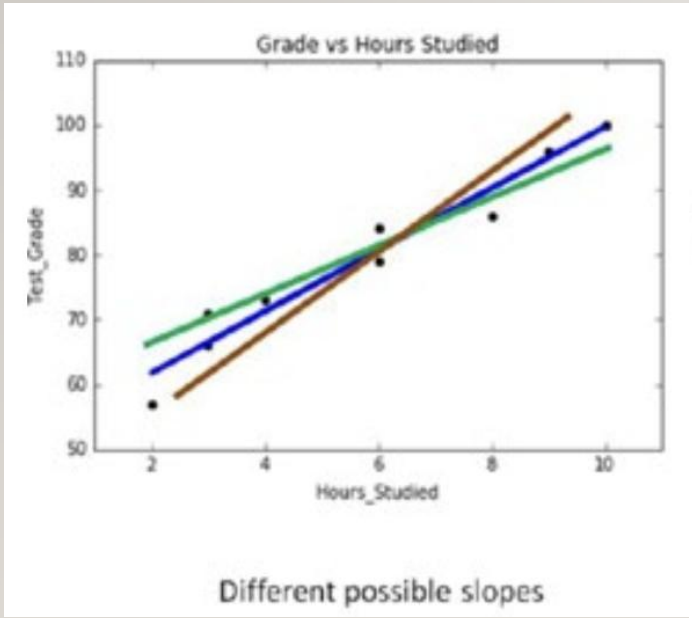
```
4                6           79
5                7           81
6                8           90
7                9           96
8               10          100
Correlation Matrix:
              Hours_Studied    Test_Grade
Hours_Studied       1.000000       0.987797
Test_Grade          0.987797       1.000000
```



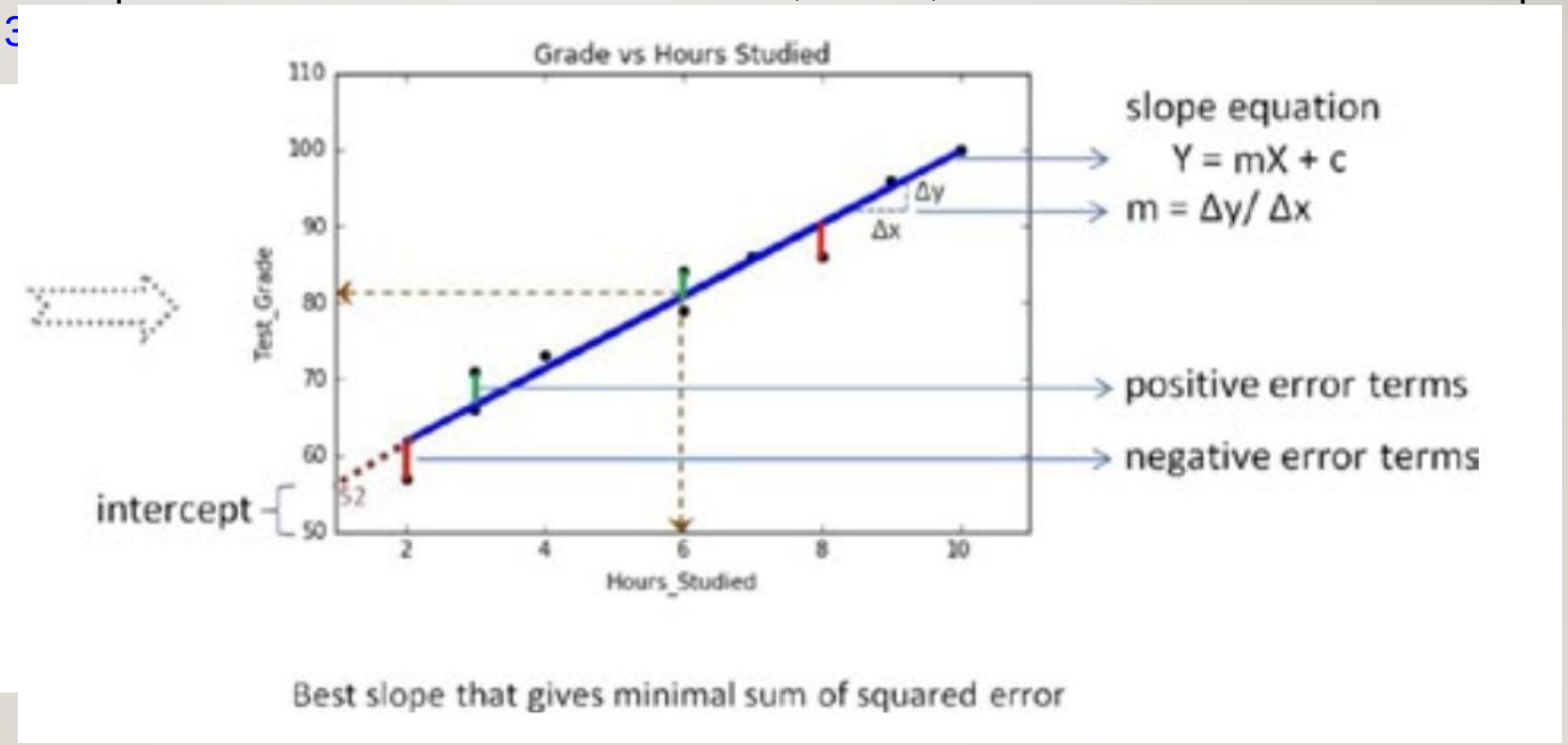
LINEAR REGRESSION (FITTING A SLOPE)

- Fitting a Slope

Let's try to fit a slope line through all the points such that the error or residual, that is, the distance of line from each point is the best possible minimal. See Figure 3



Different possible slopes

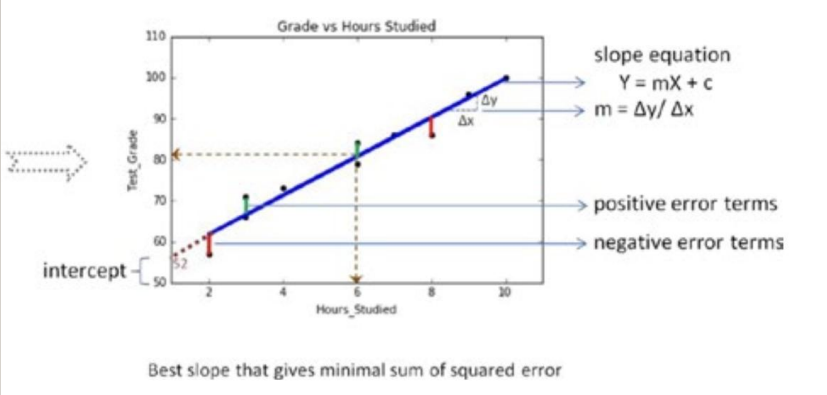


Best slope that gives minimal sum of squared error

LINEAR REGRESSION (FITTING A SLOPE)

- Fitting a Slope

Let's try to fit a slope line through all the points such that the error or residual, that is, the distance of line from each point is the best possible minimal. See Figure 3-5.



Best slope that gives minimal sum of squared error

$$m = \frac{n\sum xy - \sum x \sum y}{n\sum x^2 - (\sum x)^2}$$

$$b = \frac{\sum y - m\sum x}{n}$$

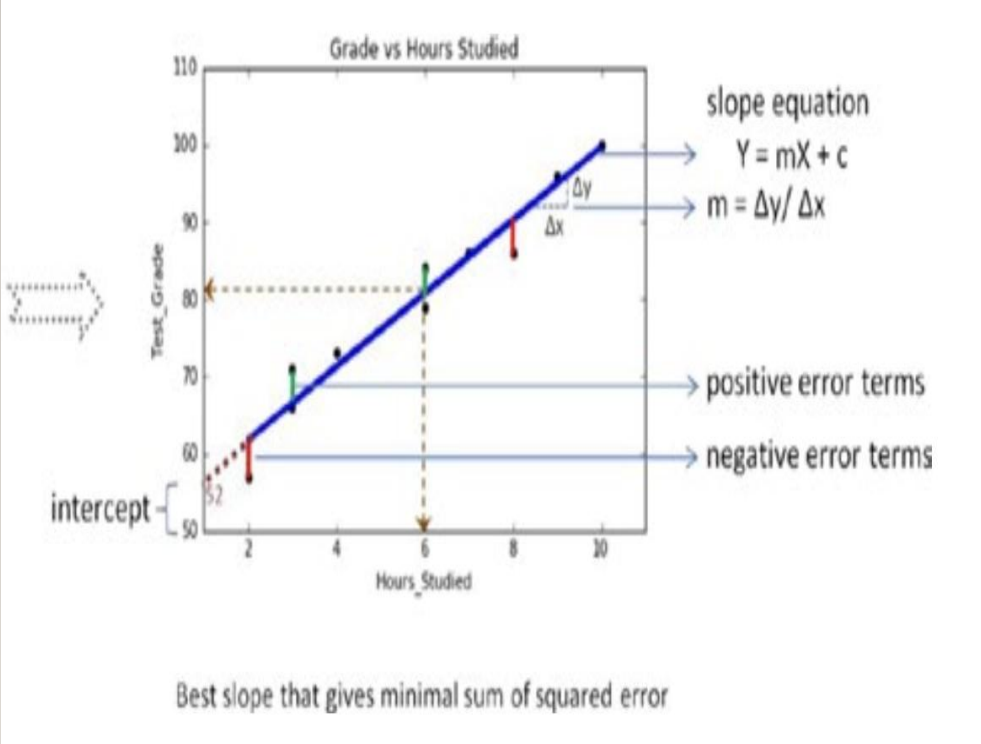
Slope: Slope is what tells you how much your target variable will change as the independent variable increases or decreases. The formula of the slope is y=mx+b

$$m = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

intercept $b = \bar{y} - a\bar{x}$

LINEAR REGRESSION (FITTING A SLOPE)

- Fitting a



Best slope that gives minimal sum of squared error

The error could be positive or negative based on its location from the slope, because of which if we take a simple sum of all the errors, it will be zero. So we should square the error to get rid of negativity and then sum the squared error. Hence, the slope is also referred to as least squares line.

•The slope equation is given by Y = mX + c, where Y is the predicted value for a given x value.

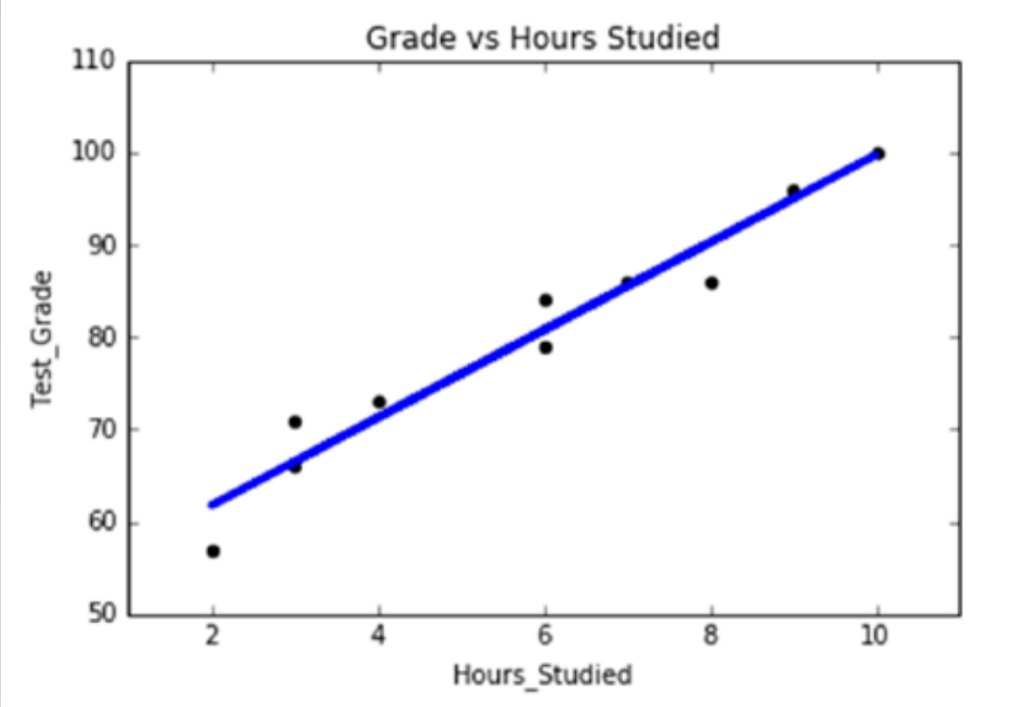
• m is the change in y, divided by change in x, that is, m is the slope of the line for the x variable and it indicates the steepness at which it increases with every unit increase in x variable value.

• c is the intercept that indicates the location or point on the axis where it intersects, in the case of Figure 3-5 it is 52.

•Intercept is a constant that represents the variability in Y that is not explained by the X. It is the value of Y when X is zero.

LINEAR REGRESSION (FITTING A SLOPE)

- Fitting a



Let's put the appropriate values in the slope equation (m * X + c = Y), 4.74260355 * 6 + 52.2928994083 = 80.74 that means a student studying 6 hours has the probability of scoring 80.74 test grade.

LINEAR REGRESSION

- How Good Is Your Model?

  There are three metrics widely used for evaluating linear model performance.

  - R-squared
  - RMSE (Root Mean Square Error)
  - MAE (Mean Absolute Error)

LINEAR REGRESSION (R-SQUARED)

- **The R-squared metric is the most popular practice of evaluating how well your model fits the data.**
- **R-squared value designates the total proportion of variance in the dependent variable explained by the independent variable.**
- **It is a value between 0 and 1; the value toward 1 indicates a better model fit. See Table 3-7.**

Table 3-7. *Sample table for R-squared calculation*

	y	$\hat{y}$	$(y_i - \bar{y})^2$	$\sum(\hat{y}_i - \bar{y})^2$
			↓	↓
Hours_Studied	Test_Grade	Test_Grade_Pred	SST	SSR
2	57	59.71111	518.8272	402.6711
3	66	64.72778	189.8272	226.5025
4	73	69.74444	45.93827	100.6678
5	76	74.76111	14.2716	25.16694
6	79	79.77778	0.604938	0
7	81	84.79444	1.493827	25.16694
8	90	89.81111	104.4938	100.6678
9	96	94.82778	263.1605	226.5025
10	100	99.84444	408.9383	402.6711
		↓		
Mean ($\bar{y}$) =		79.77		

Where,

y dependent variable

$\hat{y}$ predicted variable

$\bar{y}$ mean of dependent variable

y_i i^{th} value of dependent variable column

$\hat{y}_i$ i^{th} value of predicted dependent variable column

$$\text{R-squared} = \frac{\text{Total Sum of Square Residual } (\Sigma \text{ SSR})}{\text{Sum of Square Total}(\Sigma \text{ SST})}$$

R-squared = 1510.01 / 1547.55 = 0.97

LINEAR REGRESSION (RMSE)

- **This is the square root of the mean of the squared errors.**
- **RMSE indicates how close the predicted values are to the actual values; hence a lower RMSE value signifies that the model performance is good.**
- **One of the key properties of RMSE is that the unit will be the same as the target variable.**

Table 3-7. *Sample table for R-squared calculation*

	y	$\hat{y}$	$(y_i - \bar{y})^2$	$\sum(\hat{y}_i - \bar{y})^2$
			↓	↓
Hours_Studied	Test_Grade	Test_Grade_Pred	SST	SSR
2	57	59.71111	518.8272	402.6711
3	66	64.72778	189.8272	226.5025
4	73	69.74444	45.93827	100.6678
5	76	74.76111	14.2716	25.16694
6	79	79.77778	0.604938	0
7	81	84.79444	1.493827	25.16694
8	90	89.81111	104.4938	100.6678
9	96	94.82778	263.1605	226.5025
10	100	99.84444	408.9383	402.6711
		↓		
Mean ($\bar{y}$) =		79.77		

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2}$$

Root Mean Squared Error: 2.042299599

LINEAR REGRESSION (MAE)

- This is the mean or average of absolute value of the errors, that is, the predicted - actual.

Table 3-7. *Sample table for R-squared calculation*

	y	$\hat{y}$	$(y_i - \bar{y})^2$	$\sum(\hat{y}_i - \bar{y})^2$
			↓	↓
Hours_Studied	Test_Grade	Test_Grade_Pred	SST	SSR
2	57	59.71111	518.8272	402.6711
3	66	64.72778	189.8272	226.5025
4	73	69.74444	45.93827	100.6678
5	76	74.76111	14.2716	25.16694
6	79	79.77778	0.604938	0
7	81	84.79444	1.493827	25.16694
8	90	89.81111	104.4938	100.6678
9	96	94.82778	263.1605	226.5025
10	100	99.84444	408.9383	402.6711
		↓		
Mean ($\bar{y}$) =		79.77		

$$\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Mean Absolute Error: 1.61851851852

POLYNOMIAL REGRESSION

Machine Learning(CSE-4611)
Lecture 2

Conducted By:

Dr. Amran Hossain

Professor

CSE, DUET

Polynomial Regression

- It is a form of higher-order linear regression modeled between dependent and independent variables as an nth degree polynomial. Although it's linear it can fit curves better.
- Polynomial Regression is a regression algorithm that models the relationship between a dependent(y) and independent variable(x) as nth degree polynomial.

Features:

- ✓ It is also called the special case of Multiple Linear Regression in ML. Because we add some polynomial terms to the Multiple Linear regression equation to convert it into Polynomial Regression.
- ✓ It is a linear model with some modification in order to increase the accuracy.
- ✓ The dataset used in Polynomial regression for training is of non-linear nature.
- ✓ It makes use of a linear regression model to fit the complicated and non-

Polynomial Regression

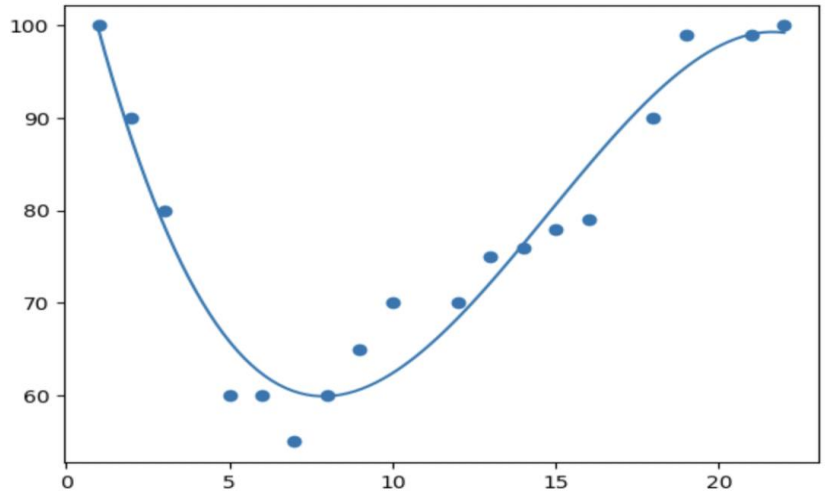
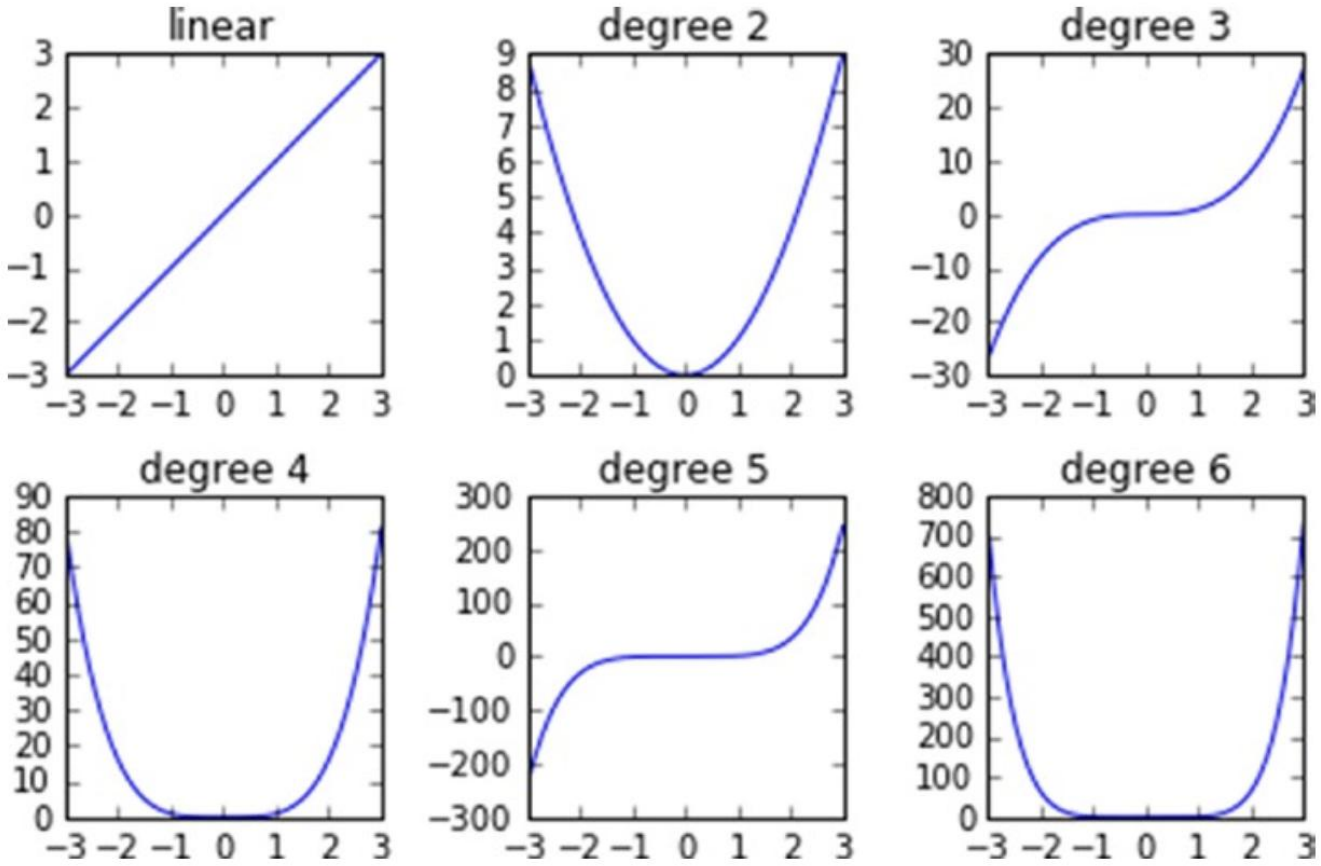


Table 3-8. *Polynomial regression higher degrees*

Degree	Regression Equation
Quadratic (2)	$Y = m_1X + m_2X^2+c$
Cubic (3)	$Y = m_1X + m_2X^2 +m_3X^3+c$
Nth	$Y = m_1X + m_2X^2 + m_3X^3 + ... +m_nX^n + c$



Polynomial Regression

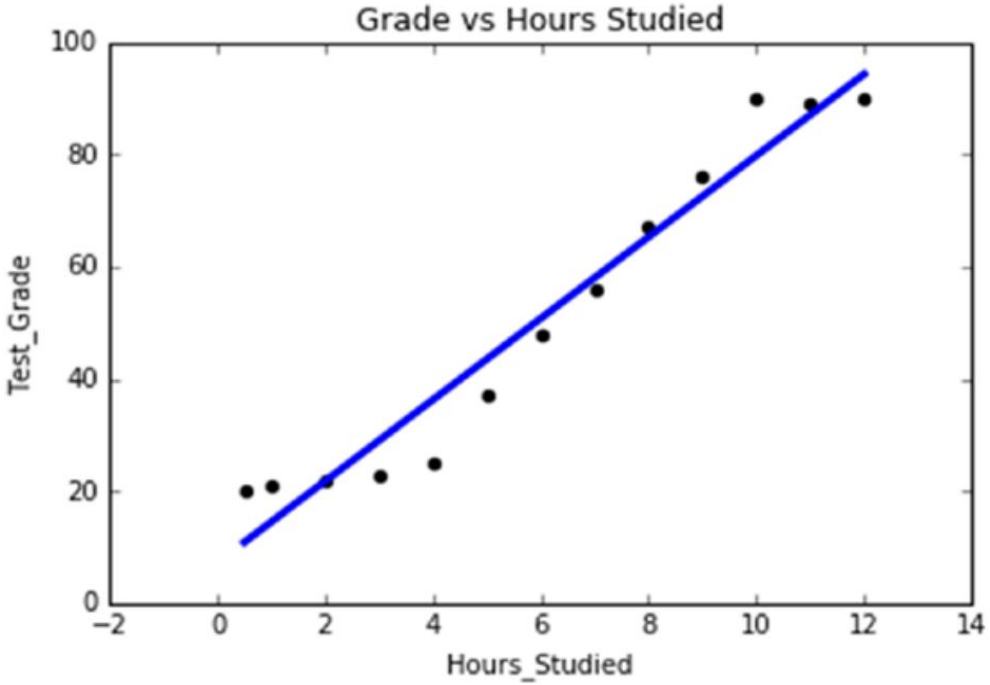
Let's consider another set of students' average test grade scores and their respective average number of hours studied for similar IQ students.

	Hours_Studied	Test_Grade
0	0.5	20
1	1.0	21
2	2.0	22
3	3.0	23
4	4.0	25
5	5.0	37
6	6.0	48
7	7.0	56
8	8.0	67
9	9.0	76
10	10.0	90
11	11.0	89
12	12.0	90

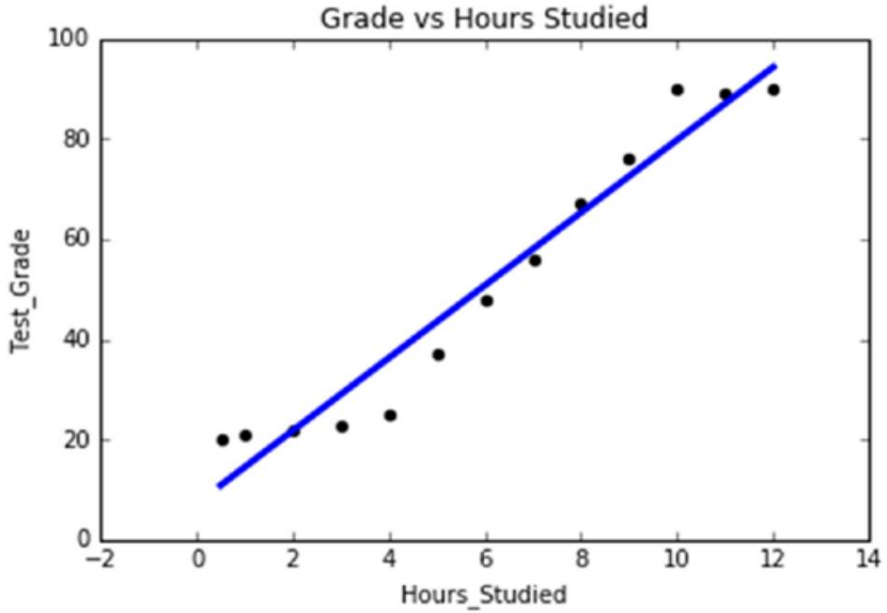
Correlation Matrix:

	Hours_Studied	Test_Grade
Hours_Studied	1.000000	0.974868
Test_Grade	0.974868	1.000000

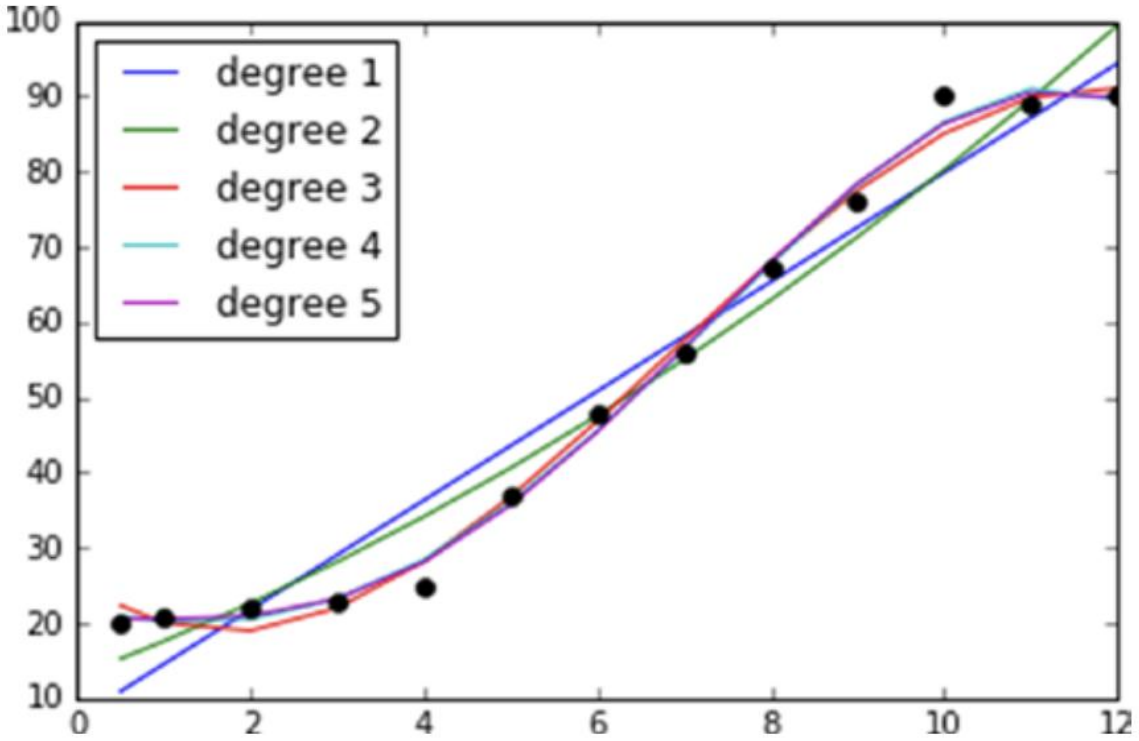
R Squared: 0.9503677767



Polynomial Regression



- ✓ The correlation analysis shows a 97% positive relationship between hours studied and the test grade, and 95% (r-squared) of variation in test grade can be explained by hours studied.
- ✓ Note that up to 4 hours of average study results in less than a 30 test grade and post 9 hours of study there is not a grade value to add to the grade.
- ✓ This is not a perfect linear relationship, although we can fit a linear line. Let's try higher-order polynomial degrees.



- ✓ Note degree 1 here is the linear fit, and the higher-order polynomial regression is fitting the curve better and r-square jumps 4% higher at degree 3.
- ✓ Beyond the 3rd degree there is not a massive change in r-squared so we can say that the 3rd degree fits better.

Multivariate Regression

- In most of the real-life use cases there will be more than one independent variable, so the concept of having multiple independent variables is called multivariate regression. The equation takes the form below.

$$y = m_1x_1 + m_2x_2 + m_3x_3 + \ldots + m_nx_n$$

- Here, each independent variable is represented by x's, and m's are the corresponding coefficients.

Multivariate Regression

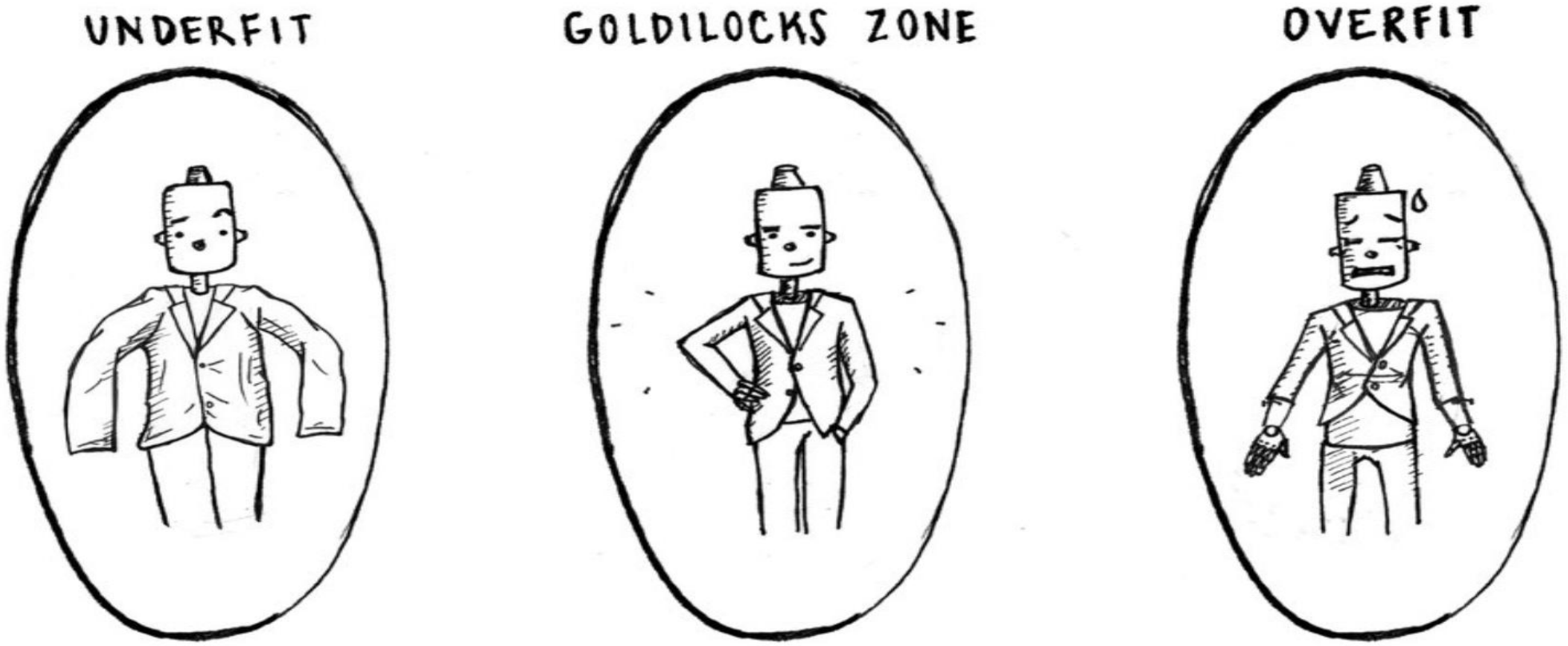
The categorical variables need to be handled appropriately before running the first iteration of the model. Scikit-learn provides useful built-in preprocessing functions to handle categorical variables.

- Label Binarizer: This will replace the binary variable text with numeric vales. We'll be using this function for the binary categorical variables.
- Label Encoder: This will replace category level with number representation.
- One Hot Encoder: This will convert n levels to n-1 new variable, and the new variables will use 1 to indicate the presence of level and 0 for otherwise. Note that before calling OneHotEncoder, we should use LabelEncoder to convert levels to number.

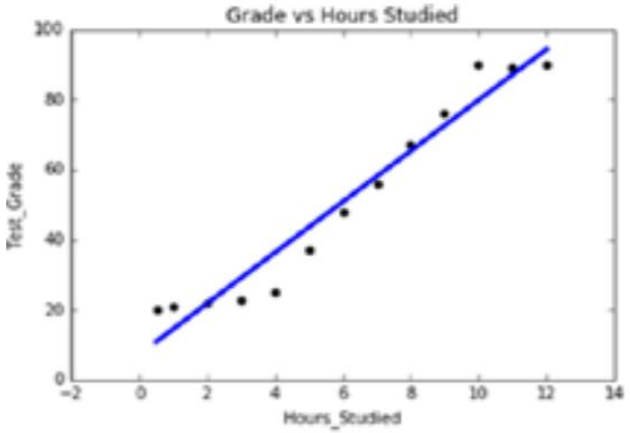
Over-fitting and Under-fitting

- Under-fitting occurs when the model does not fit the data well and is unable to capture the underlying trend in it. In this case we can notice a low accuracy in training and test dataset.
- To the contrary, over-fitting occurs when the model fits the data too well, capturing all the noises. In this case we can notice a high accuracy in the training dataset, whereas the same model will result in a low accuracy on the test dataset.
- This means the model has fitted the line so well to the train dataset that it failed to generalize it to fit well on an unseen dataset.

Over-fitting and Under-fitting

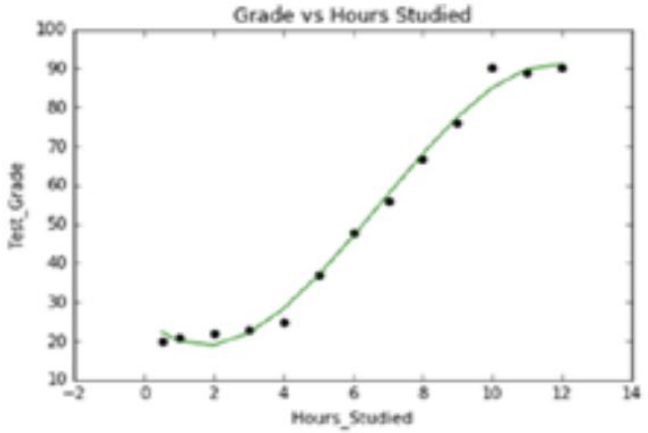


Over-fitting and Under-fitting



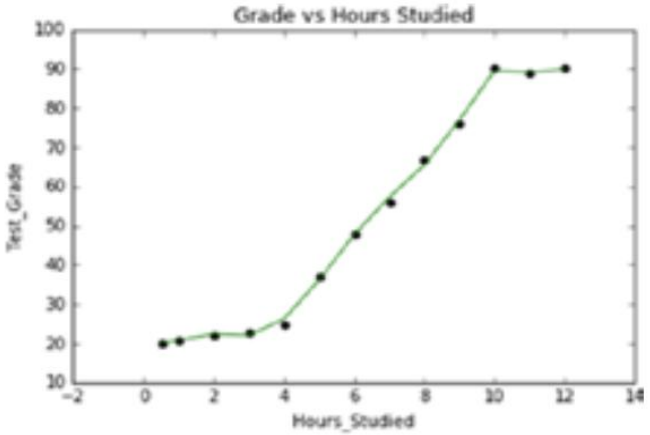
(Degree 1)

Under-fitting



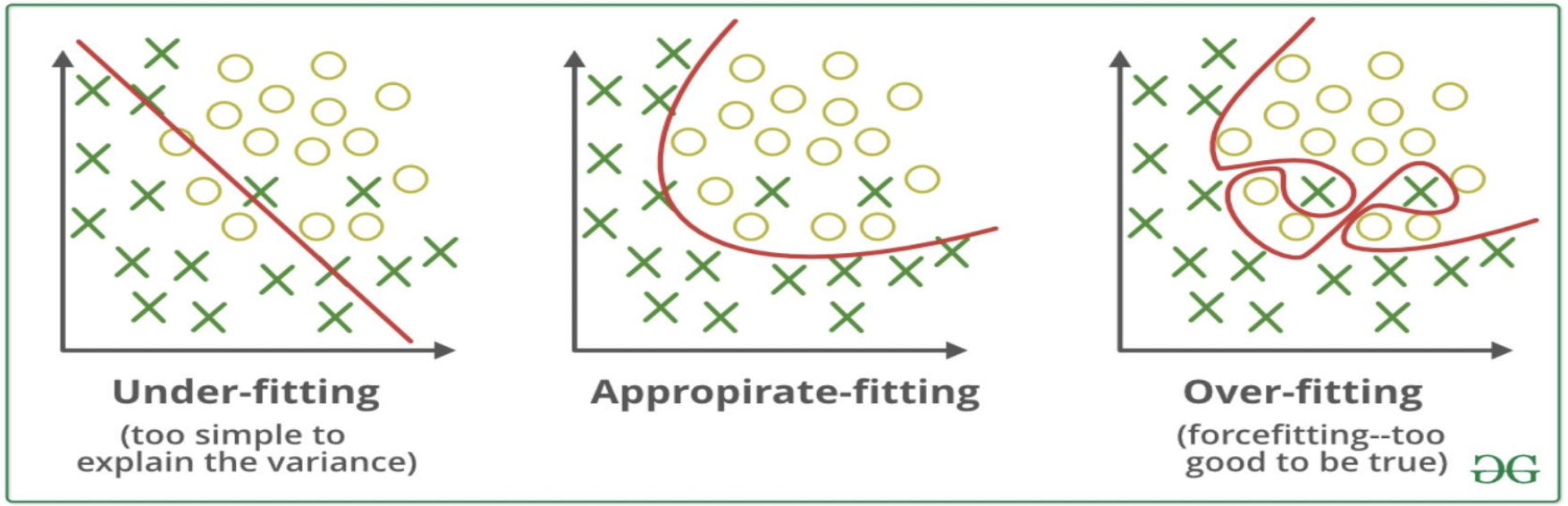
(Degree 3)

Right-fitting



(Degree 10)

Over-fitting



Regularization

- With an increase in number of variables, and increase in model complexity, the probability of over-fitting also increases. Regularization is a technique to avoid the over-fitting problem.
- Statsmodel and the scikit-learn provides Ridge and LASSO (Least Absolute Shrinkage and Selection Operator) regression to handle the over-fitting issue. With an increase in model complexity, the size of coefficients increase exponentially, so the ridge and LASSO regression apply penalty to the magnitude of the coefficient to handle the issue.

LASSO and Ridge Regression

- Lasso Regression: This provides a sparse solution, also known as L1 regularization. It guides parameter value to be zero, that is, the coefficients of the variables that add minor value to the model will be zero, and it adds a penalty equivalent to absolute value of the magnitude of coefficients.
- Ridge Regression: Also known as Tikhonov (L2) regularization, it guides parameters to be close to zero, but not zero. You can use this when you have many variables that add minor value to the model accuracy individually; however it improves overall the model accuracy and cannot be excluded from the model.
- Ridge regression will apply a penalty to reduce the magnitude of the coefficient of all variables that add minor value to the model accuracy, and which adds penalty equivalent to square of the magnitude of coefficients. Alpha is the regularization strength and must be a positive float.

What is Lasso Regression?

- Lasso regression (short for “Least Absolute Shrinkage and Selection Operator”) is a type of linear regression that is used for feature selection and regularization. Adding a penalty term to the cost function of the linear regression model is a technique used to prevent overfitting.
- The lasso regression model can be particularly useful when dealing with high-dimensional datasets, where the number of variables or features is much larger than the number of observations.
- It prevents under fitting of the Data, which is a problem in the analysis of the data.

What is Lasso Regression?

- **Mathematical equation of Lasso Regression**
- Lasso regression optimizes the following:

  Lasso regression = RSS + α * (sum of absolute value of coefficients)

  Here, α works similar to that of ridge and provides a trade-off between balancing RSS and magnitude of coefficients.

  - α = 0, Same coefficients as simple linear regression.
  - α = ∞, All coefficients zero.
  - 0 < α < ∞, coefficients between 0 and that of simple linear regression.

Ridge Regression

- Self Study

Lecture 3

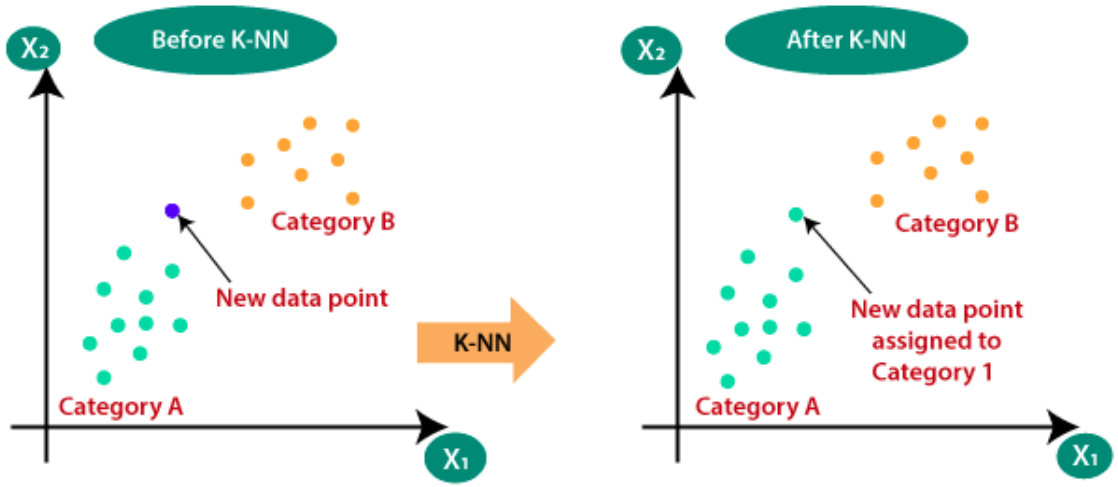
Conducted by:
Dr. Amran Hossain
Professor
CSE, DUET

Classification(KNN)

- K-Nearest Neighbor(KNN) Algorithm for Machine Learning
  - K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
  - K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
  - K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K-NN algorithm.
  - K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
  - K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
  - It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
  - KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

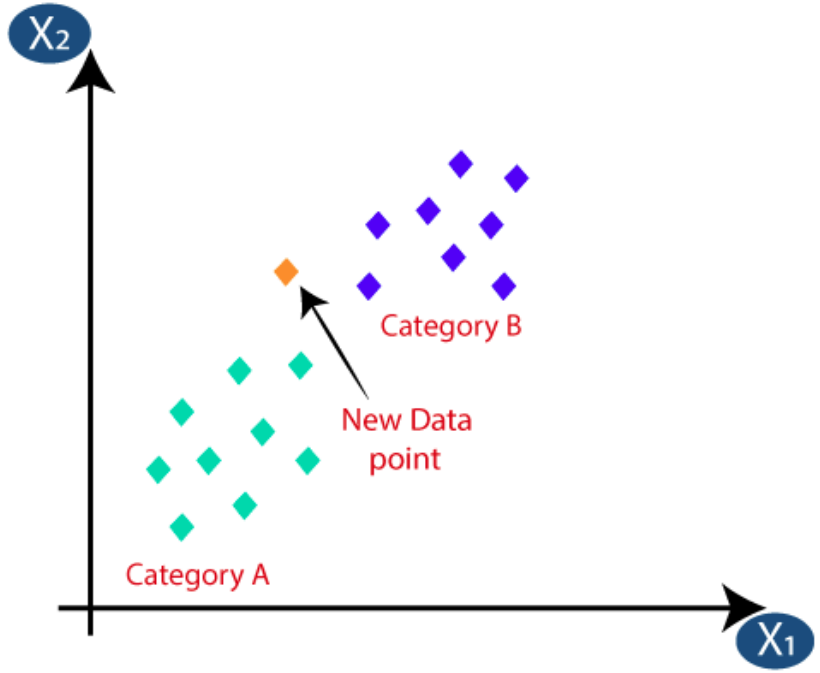
Why do we need a K-NN Algorithm?

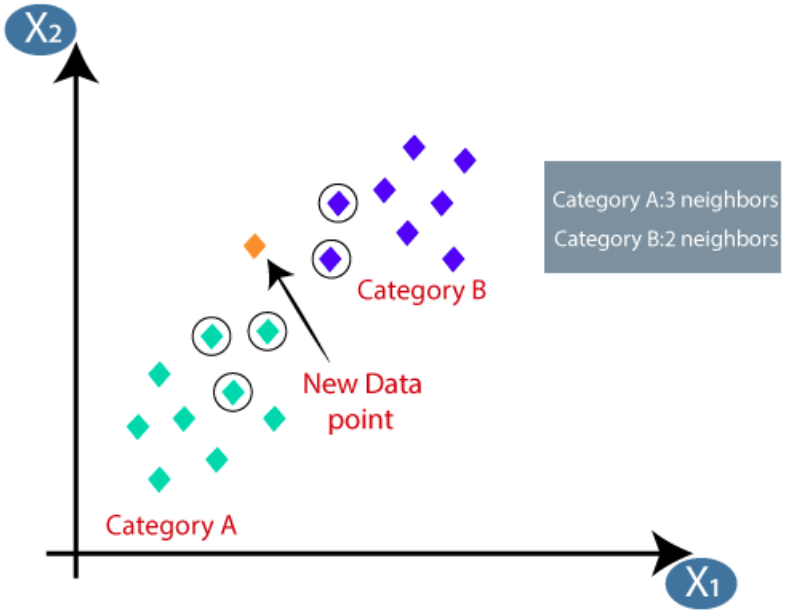
- Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x1, so this data point will lie in which of these categories.
- To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



How does K-NN work?

- The K-NN working can be explained on the basis of the below algorithm:
  - **Step-1:** Select the number K of the neighbors
  - **Step-2:** Calculate the Euclidean distance of **K number of neighbors**
  - **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
  - **Step-4:** Among these k neighbors, count the number of the data points in each category.
  - **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.





How to select the value of K in the K-NN Algorithm?

- Below are some points to remember while selecting the value of K in the K-NN algorithm:
  - There is no particular way to determine the best value for "K", so we need to try some values to find the best out of them. The most preferred value for K is 5.
  - A very low value for K such as K=1 or K=2, can be noisy and lead to the effects of outliers in the model.
  - Large values for K are good, but it may find some difficulties.

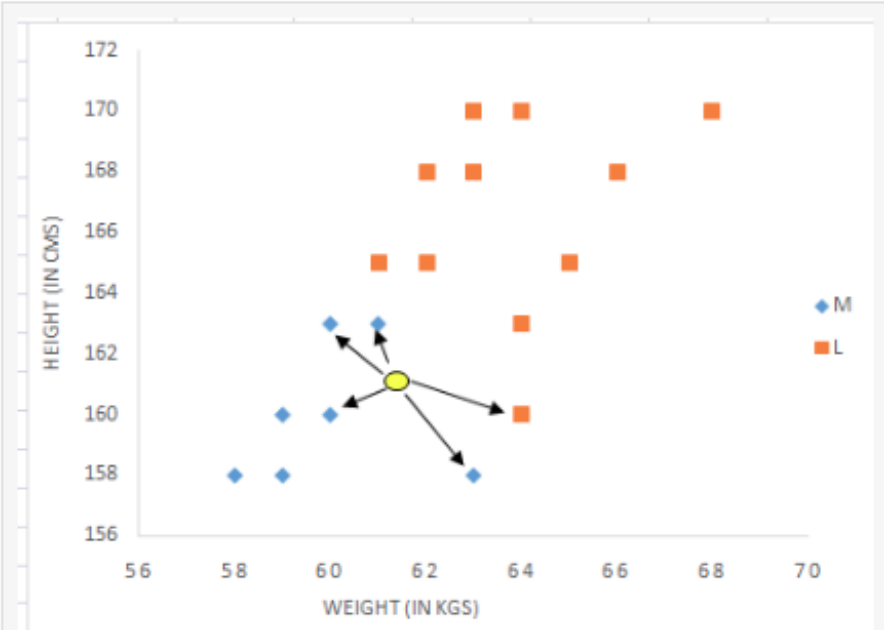
Advantages and Disadvantages of KNN

- Advantages of KNN Algorithm:
  - It is simple to implement.
  - It is robust to the noisy training data
  - It can be more effective if the training data is large.
- Disadvantages of KNN Algorithm:
  - Always needs to determine the value of K which may be complex some time.
  - The computation cost is high because of calculating the distance between the data points for all the training samples.

KNN Example

- Suppose we have height, weight and T-shirt size of some customers and we need to predict the T-shirt size of a new customer given only height and weight information we have.
- Data including height, weight and T-shirt size information is shown below -

Height (in cms)	Weight (in kgs)	T Shirt Size
158	58	M
158	59	M
158	63	M
160	59	M
160	60	M
163	60	M
163	61	M
160	64	L
163	64	L
165	61	L
165	62	L
165	65	L
168	62	L
168	63	L
168	66	L
170	63	L
170	64	L
170	68	L



KNN: Visual Representation

KNN Example

Height (in cms)	Weight (in kgs)	T Shirt Size
158	58	M
158	59	M
158	63	M
160	59	M
160	60	M
163	60	M
163	61	M
160	64	L
163	64	L
165	61	L
165	62	L
165	65	L
168	62	L
168	63	L
168	66	L
170	63	L
170	64	L
170	68	L

The idea to use distance measure is to find the distance (similarity) between new sample and training cases and then finds the k-closest customers to new customer in terms of height and weight. **New customer named 'Monica' has height 161cm and weight 61kg.** Euclidean distance between first observation and new observation (monica) is as follows -

New data point: (161, 61)

Euclidean :

$$d(x,y) = \sqrt{\sum_{i=1}^{m}(x_i - y_i)^2}$$

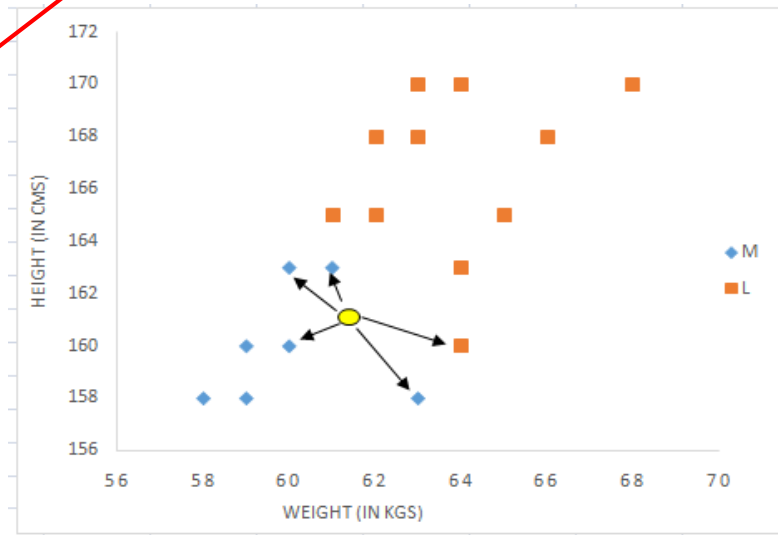
Manhattan / city - block :

$$d(x,y) = \sum_{i=1}^{m}|x_i - y_i|$$

=SQRT((161-158)^2+(61-58)^2) =4.2

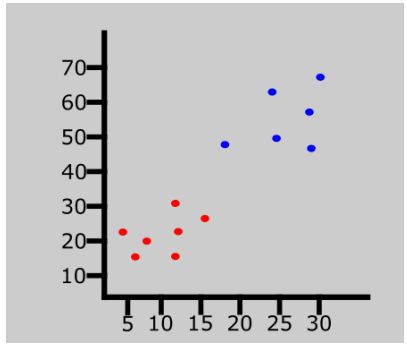
	Height (in cms)	Weight (in kgs)	T Shirt Size	Distance	
1					
2	158	58	M	4.2	
3	158	59	M	3.6	
4	158	63	M	3.6	
5	160	59	**M**	2.2	**3**
6	160	60	**M**	1.4	**1**
7	163	60	**M**	2.2	**3**
8	163	61	**M**	2.0	**2**
9	160	64	**L**	3.2	**5**
10	163	64	L	3.6	
11	165	61	L	4.0	
12	165	62	L	4.1	
13	165	65	L	5.7	
14	168	62	L	7.1	
15	168	63	L	7.3	
16	168	66	L	8.6	
17	170	63	L	9.2	
18	170	64	L	9.5	
19	170	68	L	11.4	
20					
21	**161**	**61**			

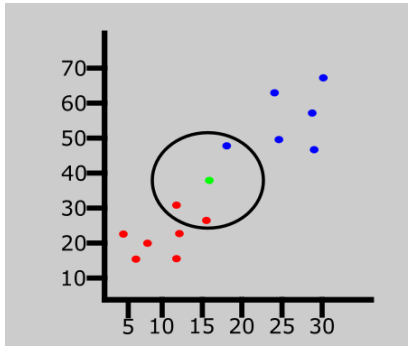
Rank (Lets K=5)

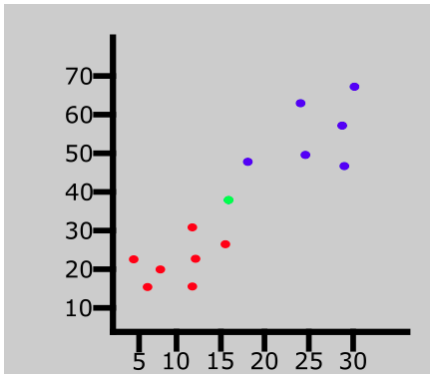


New data points

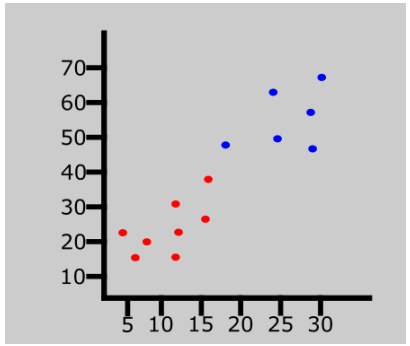
KNN Example







Since the value of **K** is 3, the algorithm will only consider the 3 nearest neighbors to the green point (new entry). This is represented in the graph above.



Dataset

BRIGHTNESS	SATURATION	CLASS
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue

New data point

BRIGHTNESS	SATURATION	CLASS
20	35	?

Here's the formula: $\sqrt{(X_2-X_1)^2+(Y_2-Y_1)^2}$

Here's what the table will look like after all the distances have been calculated:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
40	20	Red	25
50	50	Blue	33.54
60	90	Blue	68.01
10	25	Red	10
70	70	Blue	61.03
60	10	Red	47.17
25	80	Blue	45

Let's rearrange the distances in ascending order:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
10	25	Red	10
40	20	Red	25
50	50	Blue	33.54
25	80	Blue	45
60	10	Red	47.17
70	70	Blue	61.03
60	90	Blue	68.01

Since we chose 5 as the value of **K**, we'll only consider the first five rows. That is:

BRIGHTNESS	SATURATION	CLASS	DISTANCE
10	25	Red	10
40	20	Red	25
50	50	Blue	33.54
25	80	Blue	45
60	10	Red	47.17

BRIGHTNESS	SATURATION	CLASS
40	20	Red
50	50	Blue
60	90	Blue
10	25	Red
70	70	Blue
60	10	Red
25	80	Blue
20	35	Red

Classification table

Naïve Bayes Classifier Algorithm

- Naïve Bayes algorithm is a supervised learning algorithm, which is based on **Bayes theorem** and used for solving classification problems.
- It is mainly used in *text classification* that includes a high-dimensional training dataset.
- Naïve Bayes Classifier is one of the simple and most effective Classification algorithms which helps in building the fast machine learning models that can make quick predictions.
- **It is a probabilistic classifier, which means it predicts on the basis of the probability of an object**.
- Some popular examples of Naïve Bayes Algorithm are **spam filtration, Sentimental analysis, and classifying articles**.

Naïve Bayes Classifier Algorithm

- **Why is it called Naïve Bayes?**
  - The Naïve Bayes algorithm is comprised of two words Naïve and Bayes, Which can be described as:
  - **Naïve**: It is called Naïve because it assumes that the occurrence of a certain feature is independent of the occurrence of other features. Such as if the fruit is identified on the bases of color, shape, and taste, then red, spherical, and sweet fruit is recognized as an apple. Hence each feature individually contributes to identify that it is an apple without depending on each other.
  - **Bayes**: It is called Bayes because it depends on the principle of Bayes' Theorem.

Naïve Bayes Classifier Algorithm

- **Bayes' Theorem:**
  - Bayes' theorem is also known as **Bayes' Rule** or **Bayes' law**, which is used to determine the probability of a hypothesis with prior knowledge. It depends on the conditional probability.
  - The formula for Bayes' theorem is given as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Where,

P(A|B) is Posterior probability: Probability of hypothesis A on the observed event B.

P(B|A) is Likelihood probability: Probability of the evidence given that the probability of a hypothesis is true.

P(A) is Prior Probability: Probability of hypothesis before observing the evidence.

P(B) is Marginal Probability: Probability of Evidence.

Naïve Bayes Classifier Algorithm

- **Working of Naïve Bayes' Classifier:**
  1. Convert the given dataset into frequency tables.
  2. Generate Likelihood table by finding the probabilities of given features.
  3. Now, use Bayes theorem to calculate the posterior probability.

Naïve Bayes Classifier Algorithm (Example)

- Suppose we have a dataset of **weather conditions** and corresponding target variable "**Play**". So using this dataset we need to decide that whether we should play or not on a particular day according to the weather conditions.

- **Problem**: If the weather is sunny, then the Player should play or not?

Data Table

	Outlook	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

Naïve Bayes Classifier Algorithm (Example)

- Solution:
  - **Frequency table for the Weather Conditions:**

Frequency table for the Weather Conditions:

Weather	Yes	No
Overcast	5	0
Rainy	2	2
Sunny	3	2
Total	10	4

Likelihood table weather condition:

Weather	No	Yes	
Overcast	0	5	5/14= 0.35
Rainy	2	2	4/14=0.29
Sunny	2	3	5/14=0.35
All	4/14=0.29	10/14=0.71	

Naïve Bayes Classifier Algorithm (Example)

- Solution:

Applying Bayes'theorem:

P(Yes|Sunny)= P(Sunny|Yes)*P(Yes)/P(Sunny)

P(Sunny|Yes)= 3/10= 0.3

P(Sunny)= 0.35

P(Yes)=0.71

So P(Yes|Sunny) = 0.3*0.71/0.35= **0.60**

P(No|Sunny)= P(Sunny|No)*P(No)/P(Sunny)

P(Sunny|NO)= 2/4=0.5

P(No)= 0.29

P(Sunny)= 0.35

So P(No|Sunny)= 0.5*0.29/0.35 = **0.41**

So as we can see from the above calculation that **P(Yes|Sunny)>P(No|Sunny)**

Hence on a Sunny day, Player can play the game.

Naïve Bayes Classifier Algorithm (Example)

- Dataset

	Outlook	Temperature	Humidity	Windy	Play Golf
0	Rainy	Hot	High	False	No
1	Rainy	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Sunny	Mild	High	False	Yes
4	Sunny	Cool	Normal	False	Yes
5	Sunny	Cool	Normal	True	No
6	Overcast	Cool	Normal	True	Yes
7	Rainy	Mild	High	False	No
8	Rainy	Cool	Normal	False	Yes
9	Sunny	Mild	Normal	False	Yes
10	Rainy	Mild	Normal	True	Yes
11	Overcast	Mild	High	True	Yes
12	Overcast	Hot	Normal	False	Yes
13	Sunny	Mild	High	True	No

Now, with regards to our dataset, we can apply Bayes' theorem in following way:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)}$$

where, y is class variable and X is a dependent feature vector (of size *n*) where:

$$X = (x_1, x_2, x_3, ..., x_n)$$

Hence, we reach to the result:

$$P(y|x_1, ..., x_n) = \frac{P(x_1|y)P(x_2|y)...P(x_n|y)P(y)}{P(x_1)P(x_2)...P(x_n)}$$

which can be expressed as:

$$P(y|x_1, ..., x_n) = \frac{P(y)\prod_{i=1}^{n} P(x_i|y)}{P(x_1)P(x_2)...P(x_n)}$$

Now, as the denominator remains constant for a given input, we can remove that term:

$$P(y|x_1, ..., x_n) \propto P(y)\prod_{i=1}^{n} P(x_i|y)$$

Naïve Bayes Classifier Algorithm (Example)

- Dataset

  Problem: X = (Rainy, Hot, High, False)

Calculate frequency and likelihood:

Outlook

	Yes	No	P(yes)	P(no)
Sunny	2	3	2/9	3/5
Overcast	4	0	4/9	0/5
Rainy	3	2	3/9	2/5
Total	9	5	100%	100%

Temperature

	Yes	No	P(yes)	P(no)
Hot	2	2	2/9	2/5
Mild	4	2	4/9	2/5
Cool	3	1	3/9	1/5
Total	9	5	100%	100%

Humidity

	Yes	No	P(yes)	P(no)
High	3	4	3/9	4/5
Normal	6	1	6/9	1/5
Total	9	5	100%	100%

Wind

	Yes	No	P(yes)	P(no)
False	6	2	6/9	2/5
True	3	3	3/9	3/5
Total	9	5	100%	100%

Play		P(Yes)/P(No)
Yes	9	9/14
No	5	5/14
Total	14	100%

Naïve Bayes Classifier Algorithm (Example)

- Dataset

  Problem: today = (Sunny, Hot, Normal, False) ??

So now, we are done with our pre-computations and the classifier is ready!
Let us test it on a new set of features (let us call it today):

today = (Sunny, Hot, Normal, False)

So, probability of playing golf is given by:

$$P(Yes|today) = \frac{P(SunnyOutlook|Yes)P(HotTemperature|Yes)P(NormalHumidity|Yes)P(NoWind|Yes)P(Yes)}{P(today)}$$

and probability to not play golf is given by:

$$P(No|today) = \frac{P(SunnyOutlook|No)P(HotTemperature|No)P(NormalHumidity|No)P(NoWind|No)P(No)}{P(today)}$$

Naïve Bayes Classifier Algorithm (Example)

Since, P(today) is common in both probabilities, we can ignore P(today) and find proportional probabilities as:

$$P(Yes|today) \propto \frac{2}{9}.\frac{2}{9}.\frac{6}{9}.\frac{6}{9}.\frac{9}{14} \approx 0.0141$$

and

$$P(No|today) \propto \frac{3}{5}.\frac{2}{5}.\frac{1}{5}.\frac{2}{5}.\frac{5}{14} \approx 0.0068$$

Naïve Bayes Classifier Algorithm (Example)

Now, since

$$P(Yes|today) + P(No|today) = 1$$

These numbers can be converted into a probability by making the sum equal to 1 (normalization):

$$P(Yes|today) = \frac{0.0141}{0.0141+0.0068} = 0.67$$

and

$$P(No|today) = \frac{0.0068}{0.0141+0.0068} = 0.33$$

Since

$$P(Yes|today) > P(No|today)$$

So, prediction that golf would be played is ‘Yes’.

Classification of Naïve Bayes

- There are four types of the Naive Bayes Model, which are explained below:
- 1) Gaussian Naïve Bayes
- 2) Optimal Naïve Bayes
- 3) Bernoulli Naïve Bayes
- 4) Multinominal Naïve Bayes

Gaussian Naïve Bayes

- It is a straightforward algorithm used when the attributes are continuous. The attributes present in the data should follow the rule of Gaussian distribution or normal distribution.
- It remarkably quickens the search, and under lenient conditions, the error will be two times greater than Optimal Naive Bayes.

BAYES DECISION THEORY

- We will initially focus on the two-class case. Let w_1, w_2 be the two classes in which our patterns belong. In the sequel, we assume that the ***a priori probabilities*** $\boldsymbol{P}(w_1)$, $\boldsymbol{P}(w_2)$ are known. This is a very reasonable assumption, because even if they are not known, they can easily be estimated from the available training feature vectors. Indeed, if $\boldsymbol{N}$ is the total number of available training patterns, and $\boldsymbol{N}_1$, $\boldsymbol{N}_2$ of them belong to w_1 and w_2, respectively, then $\boldsymbol{P}(w_1) \approx \boldsymbol{N}_1/\boldsymbol{N}$ and $\boldsymbol{P}(w_2) \approx \boldsymbol{N}_2$

We now have all the ingredients to compute our conditional probabilities, as stated in the introduction. To this end, let us recall from our probability course basics the *Bayes rule* (Appendix A)

$$P(\omega_i|\boldsymbol{x}) = \frac{p(\boldsymbol{x}|\omega_i)P(\omega_i)}{p(\boldsymbol{x})} \tag{2.1}$$

where $p(\boldsymbol{x})$ is the pdf of $\boldsymbol{x}$ and for which we have (Appendix A)

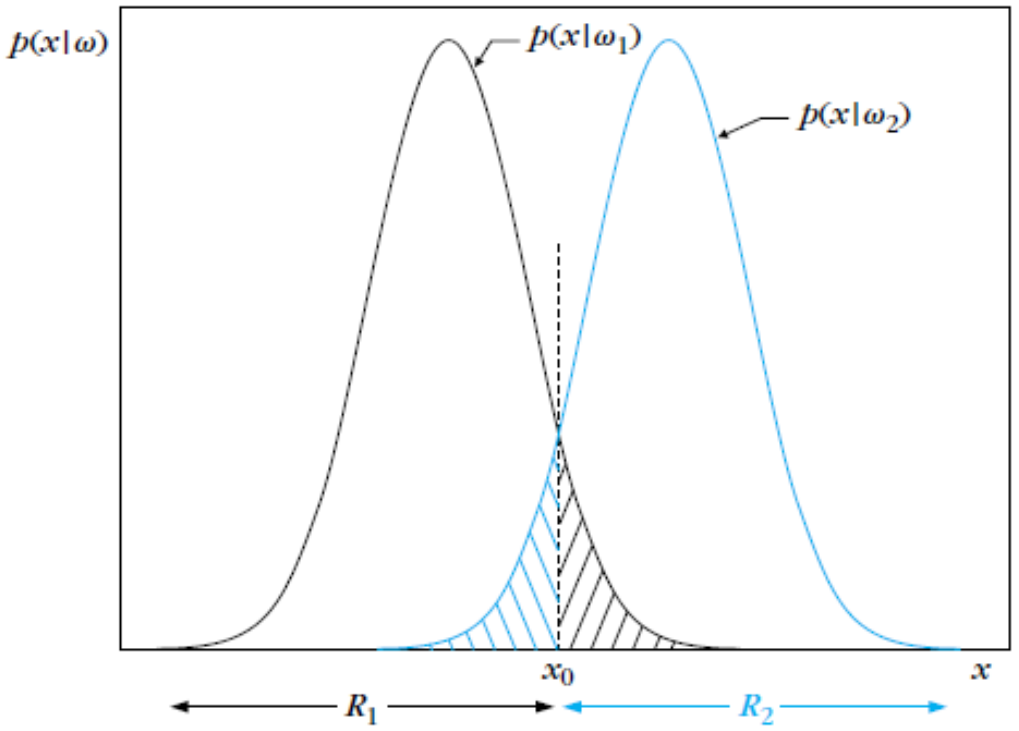
$$p(\boldsymbol{x}) = \sum_{i=1}^{2} p(\boldsymbol{x}|\omega_i)P(\omega_i) \tag{2.2}$$

The *Bayes classification rule* can now be stated as

$$\begin{aligned} &\text{If} \quad P(\omega_1|\boldsymbol{x}) > P(\omega_2|\boldsymbol{x}), \quad \boldsymbol{x} \text{ is classified to } \omega_1 \\ &\text{If} \quad P(\omega_1|\boldsymbol{x}) < P(\omega_2|\boldsymbol{x}), \quad \boldsymbol{x} \text{ is classified to } \omega_2 \end{aligned} \tag{2.3}$$

BAYES DECISION THEORY

Figure 2.1 presents an example of two equiprobable classes and shows the variations of $p(x\,w_i)$, $i = 1, 2$, as functions of x for the simple case of a single feature ($l = 1$). The dotted line at x_0 is a threshold partitioning the feature space into two regions, R_1 and R_2. According to the Bayes decision rule, for all values of x in R_1 the classifier decides w_1 and for all values in R_2 it decides w_2.



However, it is obvious from the figure that decision errors are unavoidable. Indeed, there is a finite probability for an x to lie in the R_2 region and at the same time to belong in class w_1. Then our decision is in error. The same is true for points originating from class w_2. It does not take much thought to see that the total probability, P_e, of committing a decision error for the case of two equiprobable classes, is given by

$$P_e = \frac{1}{2}\int_{-\infty}^{x_0} p(x|\omega_2)\,dx + \frac{1}{2}\int_{x_0}^{+\infty} p(x|\omega_1)\,dx \qquad (2.6)$$

which is equal to the total shaded area under the curves in Figure 2.1.

BAYES DECISION THEORY

Minimizing the Classification Error Probability:

Proof. Let R_1 be the region of the feature space in which we decide in favor of ω_1 and R_2 be the corresponding region for ω_2. Then an error is made if $\boldsymbol{x} \in R_1$, although it belongs to ω_2 or if $\boldsymbol{x} \in R_2$, although it belongs to ω_1. That is,

$$P_e = P(\boldsymbol{x} \in R_2, \omega_1) + P(\boldsymbol{x} \in R_1, \omega_2) \tag{2.7}$$

where $P(\cdot, \cdot)$ is the joint probability of two events. Recalling, once more, our probability basics (Appendix A), this becomes

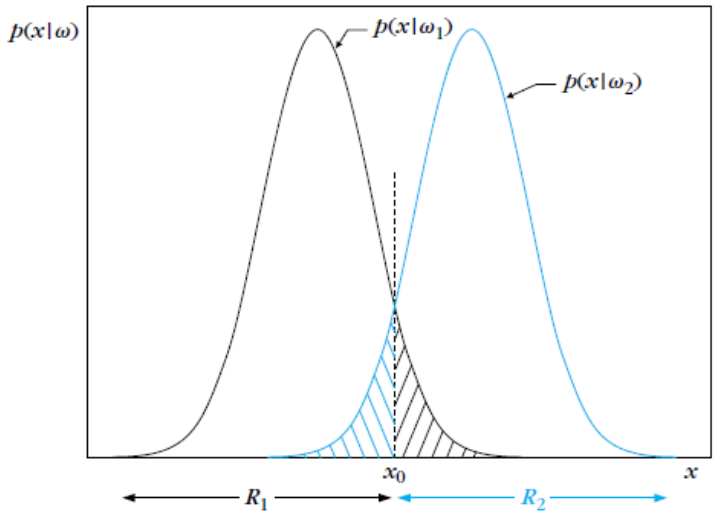
$$\begin{aligned} P_e &= P(\boldsymbol{x} \in R_2|\omega_1)P(\omega_1) + P(\boldsymbol{x} \in R_1|\omega_2)P(\omega_2) \\ &= P(\omega_1) \int_{R_2} p(\boldsymbol{x}|\omega_1)\, d\boldsymbol{x} + P(\omega_2) \int_{R_1} p(\boldsymbol{x}|\omega_2)\, d\boldsymbol{x} \end{aligned} \tag{2.8}$$

or using the Bayes rule

$$P_e = \int_{R_2} P(\omega_1|\boldsymbol{x})p(\boldsymbol{x})\, d\boldsymbol{x} + \int_{R_1} P(\omega_2|\boldsymbol{x})p(\boldsymbol{x})\, d\boldsymbol{x} \tag{2.9}$$

It is now easy to see that the error is minimized if *the partitioning regions* R_1 and R_2 *of the feature space are chosen so that*

$$\begin{aligned} R_1&: P(\omega_1|\boldsymbol{x}) > P(\omega_2|\boldsymbol{x}) \\ R_2&: P(\omega_2|\boldsymbol{x}) > P(\omega_1|\boldsymbol{x}) \end{aligned} \tag{2.10}$$



BAYES DECISION THEORY

Minimizing the Classification Error Probability:

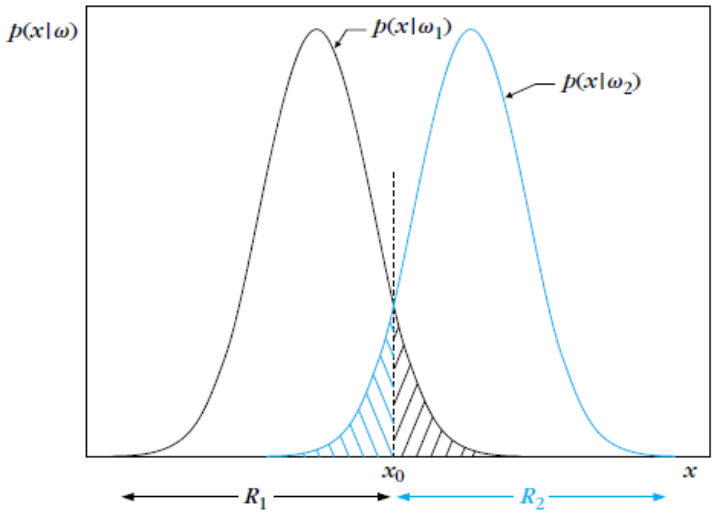
Indeed, since the union of the regions R_1, R_2 covers all the space, from the definition of a probability density function we have that

$$\int_{R_1} P(\omega_1|x)p(x)\,dx + \int_{R_2} P(\omega_1|x)p(x)\,dx = P(\omega_1) \tag{2.11}$$

Combining Eqs. (2.9) and (2.11), we get

$$P_e = P(\omega_1) - \int_{R_1} (P(\omega_1|x) - P(\omega_2|x))\,p(x)\,dx \tag{2.12}$$

This suggests that the probability of error is minimized if R_1 is the region of space in which $P(\omega_1|x) > P(\omega_2|x)$. Then, R_2 becomes the region where the reverse is true.



- **The Gaussian Probability Density Function**

The one-dimensional or the univariate Gaussian, as it is sometimes called, is defined bv

$$p(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{2.24}$$

The parameters μ and σ^2 turn out to have a specific meaning. The mean value of the random variable x is equal to μ, that is,

$$\mu = E[x] \equiv \int_{-\infty}^{+\infty} xp(x)dx \tag{2.25}$$

where $E[\cdot]$ denotes the mean (or expected) value of a random variable. The parameter σ^2 is equal to the variance of x, that is,

$$\sigma^2 = E[(x-\mu)^2] \equiv \int_{-\infty}^{+\infty} (x-\mu)^2 p(x)dx \tag{2.26}$$

- **The Gaussian Probability Density Function**

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Mean

$$\sigma = \left[\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \mu)^2\right]^{0.5}$$

Standard deviation

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Normal distribution

- **The Gaussian Probability Density Function**

		Humidity	*Mean*	*StDev*
Play Golf	yes	86 96 80 65 70 80 70 90 75	79.1	10.2
	no	85 90 70 95 91	86.2	9.7

$$P(\text{humidity} = 74 \mid \text{play} = \text{yes}) = \frac{1}{\sqrt{2\pi}(10.2)} e^{-\frac{(74-79.1)^2}{2(10.2)^2}} = 0.0344$$

$$P(\text{humidity} = 74 \mid \text{play} = \text{no}) = \frac{1}{\sqrt{2\pi}(9.7)} e^{-\frac{(74-86.2)^2}{2(9.7)^2}} = 0.0187$$

• The Gaussian Probability Density Function

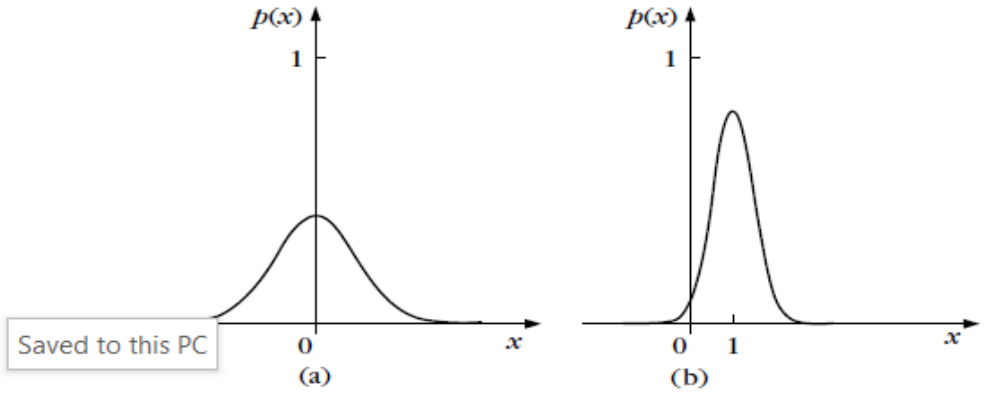


FIGURE 2.2

Graphs for the one-dimensional Gaussian pdf. (a) Mean value $\mu = 0$, $\sigma^2 = 1$, (b) $\mu = 1$ and $\sigma^2 = 0.2$. The larger the variance the broader the graph is. The graphs are symmetric, and they are centered at the respective mean value.

Figure 2.2a shows the graph of the Gaussian function for $\mu = 0$ and $\sigma^2 = 1$, and Figure 2.2b the case for $\mu = 1$ and $\sigma^2 = 0.2$. The larger the variance the broader the graph, which is symmetric, and it is always centered at μ (see Appendix A, for some more properties).

The multivariate generalization of a Gaussian pdf in the l-dimensional space is given by

$$p(\boldsymbol{x}) = \frac{1}{(2\pi)^{l/2}|\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T\Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right) \tag{2.27}$$

where $\boldsymbol{\mu} = E[\boldsymbol{x}]$ is the mean value and Σ is the $l \times l$ *covariance matrix* (Appendix A) defined as

$$\Sigma = E[(\boldsymbol{x}-\boldsymbol{\mu})(\boldsymbol{x}-\boldsymbol{\mu})^T] \tag{2.28}$$

where $|\Sigma|$ denotes the determinant of Σ. It is readily seen that for $l = 1$ the multivariate Gaussian coincides with the univariate one. Sometimes, the symbol $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ is used to denote a Gaussian pdf with mean value $\boldsymbol{\mu}$ and covariance Σ.

To get a better feeling on what the multivariate Gaussian looks like, let us focus on some cases in the two-dimensional space, where nature allows us the luxury of visualization. For this case we have

$$\Sigma = E\left[\begin{bmatrix} x_1-\mu_1 \\ x_2-\mu_2 \end{bmatrix}\begin{bmatrix} x_1-\mu_1, & x_2-\mu_2 \end{bmatrix}\right] \tag{2.29}$$

$$= \begin{bmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{12} & \sigma_2^2 \end{bmatrix} \tag{2.30}$$

- **Optimal Naive Bayes**
- Optimal Naive Bayes selects the class that has the greatest posterior probability of happenings. As per the name, it is optimal. But it will go through all the possibilities, which is very slow and time-consuming.
- **Bernoulli Naive Bayes**
- Bernoulli Naive Bayes is an algorithm that is useful for data that has binary or boolean attributes. The attributes will have a value of yes or no, useful or not, granted or rejected, etc.
- **Multinominal Naive Bayes**
- Multinominal Naive Bayes is used on documentation classification issues. The features needed for this type are the frequency of the words converted from the document.

- **Advantages of a Naive Bayes Classifier**
- Here are some advantages of the Naive Bayes Classifier:
  - It doesn’t require larger amounts of training data.
  - It is straightforward to implement.
  - Convergence is quicker than other models, which are discriminative.
  - It is highly scalable with several data points and predictors.
  - It can handle both continuous and categorical data.
  - It is not sensitive to irrelevant data and doesn’t follow the assumptions it holds.
  - It is used in real-time predictions.

- **Disadvantages of a Naive Bayes Classifier**
- The disadvantage of the Naive Bayes Classifier are as below:
  - The Naive Bayes Algorithm has trouble with the ‘zero-frequency problem’. It happens when you assign zero probability for categorical variables in the training dataset that is not available. When you use a smooth method for overcoming this problem, you can make it work the best.
  - It will assume that all the attributes are independent, which rarely happens in real life. It will limit the application of this algorithm in real-world situations.
  - It will estimate things wrong sometimes, so you shouldn’t take its probability outputs seriously.

Bernoulli Naive Bayes

- Bernoulli Naive Bayes is a subcategory of the Naive Bayes Algorithm. It is used for the classification of binary features such as ‘Yes’ or ‘No’, ‘1’ or ‘0’, ‘True’ or ‘False’ etc. Here it is to be noted that the features are independent of one another.
- Bernoulli Naive Bayes is basically used for spam detection, text classification, Sentiment Analysis, used to determine whether a certain word is present in a document or not.

Bernoulli Naive Bayes

- **Bernoulli** Distribution:-
- Let there be a random variable 'X' and let the probability of success be denoted by 'p' and the likelihood of failure be represented by 'q.'
  - Success: p
  - Failure: q

  - q = 1 - (probability of Sucesss)
  - q = 1 - p

$$p(x) = P[X = x] = \begin{cases} q = 1 - p & x = 0 \\ p & x = 1 \end{cases}$$

$$X = \begin{cases} 1 & \text{Bernoulli trial} = \mathbf{S} \\ 0 & \text{Bernoulli trial} = \mathbf{F} \end{cases}$$

Bernoulli Naive Bayes (Example)

Let us consider the example below to understand Bernoulli Naive Bayes:-

Adult	Gender	Fever	Disease
Yes	Female	No	False
Yes	Female	Yes	True
No	Male	Yes	False
No	Male	No	True
Yes	Male	Yes	True

- ✓ In the above dataset, we are trying to predict whether a person has a disease or not based on their age, gender, and fever.
- ✓ Here, 'Disease' is the target, and the rest are the features.
- ✓ All values are binary.

We wish to classify an instance 'X' where Adult='Yes', Gender= 'Male', and Fever='Yes'.

Bernoulli Naive Bayes (Example)

We wish to classify an instance 'X' where Adult='Yes', Gender= 'Male', and Fever='Yes'.

Let us consider the example below to understand Bernoulli Naive Bayes:-

Adult	Gender	Fever	Disease
Yes	Female	No	False
Yes	Female	Yes	True
No	Male	Yes	False
No	Male	No	True
Yes	Male	Yes	True

Firstly, we calculate the class probability, probability of disease or not.

P(Disease = True) = ⅗

P(Disease = False) = ⅖

Secondly, we calculate the individual probabilities for each feature.

P(Adult= Yes | Disease = True) = ⅔
P(Gender= Male | Disease = True) = ⅔
P(Fever= Yes | Disease = True) = ⅔

P(Adult= Yes | Disease = False) = ½
P(Gender= Male | Disease = False) = ½
P(Fever = Yes | Disease = False) = ½

Bernoulli Naive Bayes (Example)

We wish to classify an instance 'X' where Adult='Yes', Gender= 'Male', and Fever='Yes'.

Let us consider the example below to understand Bernoulli Naive Bayes:-

Adult	Gender	Fever	Disease
Yes	Female	No	False
Yes	Female	Yes	True
No	Male	Yes	False
No	Male	No	True
Yes	Male	Yes	True

Now, we need to find out two probabilities:-
(i) P(Disease= True | X) = (P(X | Disease= True) * P(Disease=True))/ P(X)
(ii) P(Disease = False | X) = (P(X | Disease = False) * P(Disease= False))/P(X)

P(Disease = True | X) = ((⅔ *⅔ * ⅔) * (⅗))/P(X) = (8/27 * ⅗) / P(X) = 0.17/P(X)

P(Disease = False | X) = [(½ * ½ * ½) * (⅖)] / P(X) = [⅛ * ⅖] / P(X) = 0.05/ P(X)

Now, we calculate estimator probability:-

P(X) = P(Adult= Yes) * P(Gender = Male) * P(Fever = Yes)
= ⅗ * ⅗ * ⅗ = 27/125 = 0.21

So we get finally:-
P(Disease = True | X) = 0.17 / P(X)
= 0.17 / 0.21
= 0.80 ……(1)
P(Disease = False | X) = 0.05 / P(X)
= 0.05 / 0.21
= 0.23 ……..(2)

Now, we notice that (1) > (2), the result of instance 'X' is 'True', i.e., the person has the disease.